==========

Python Basics

Author: Nagesh Sir , KL University Hyderabad

Objective: Learn Python fundamentals for Data Science - Part 1

==========

In [31]:
```
# ============================
#  STEP 1: Hello World
# ============================
print("Hello, Python!")
print(10 + 5)
```

```
Hello, Python!
15
```

In [3]:
```
#============================
# STEP 2: Variables
# ============================
a = 10
b = 20
print("Value of a:", a)
print("Value of b:", b)
```

```
Value of a: 10
Value of b: 20
```

In [5]:
```
name = "KL University"
print("Name:", name)
```

```
Name: KL University
```

In [1]:
```
# ============================
#  STEP 3: Data Types
# ============================
x = 10
y = 10.5
text = "Python"
flag = True


print(type(x))
print(type(y))
print(type(text))
print(type(flag))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'list'>
```

In [12]:
```python
# ============================
#  STEP 4: User Input
# ============================
# Uncomment to test
num = int(input("Enter a number: "))
print("You entered:", num)
```

```
Enter a number: 2
You entered: 2
```

In [13]:
```python
# ============================
#  STEP 5: Conditional Statements
# ============================
num = 15

if num > 10:
    print("Greater than 10")
else:
    print("Less than or equal to 10")
```

```
Greater than 10
```

In [14]:
```python
# ============================
#  STEP 6: Loops
# ============================
print("For Loop:")
for i in range(5):
    print(i)
```

```
For Loop:
0
1
2
3
4
```

In [15]:
```python
print("While Loop:")
i = 1
while i <= 5:
    print(i)
    i += 1
```

```
While Loop:
1
2
3
4
5
```

In [16]:
```python
# ============================
#  STEP 7: Lists
# ============================
numbers = [10, 20, 30, 40]
print(numbers[0])
```

```
10
```

In [17]:
```python
numbers.append(50)
```

In [18]:
```
for num in numbers:
    print(num)
```

```
10
20
30
40
50
```

In [22]:
```
# ============================
#  STEP 8: Dictionaries
# ============================
student = {
    "name": "Hanish",
    "age": 21,
    "marks": 90
}
```

In [23]:
```
print(student["name"])

student["grade"] = "A"
print(student)
```

```
Hanish
{'name': 'Hanish', 'age': 21, 'marks': 90, 'grade': 'A'}
```

In [24]:
```
# ============================
#  STEP 9: Functions
# ============================
def add(a, b):
    return a + b
```

In [25]:
```
result = add(10, 20)
print("Sum:", result)
```

```
Sum: 30
```

In [26]:
```
# ============================
#  STEP 10: Programs
# ============================
# Even or Odd
num = 7
if num % 2 == 0:
    print("Even")
else:
    print("Odd")
```

```
Odd
```

In [27]:
```
# Sum of numbers
total = 0
for i in range(1, 6):
    total += i
print("Sum:", total)
```

```
Sum: 15
```

In [28]:
```
# Largest number
a = 10
b = 25
c = 15

if a > b and a > c:
    print("A is largest")
```

```
elif b > c:
    print("B is largest")
else:
    print("C is largest")
```

```
B is largest
```

In [30]:
```
print("Lab Completed Successfully!")
```

```
Lab Completed Successfully!
```

==========

NUMPY LAB (Python for Data Science - Part 2)

==========

In [32]:
```
 #STEP 1: Import NumPy
import numpy as np
```

In [33]:
```
#  STEP 2: Create Arrays
arr1 = np.array([1, 2, 3, 4, 5])
print("Array:", arr1)
```

```
Array: [1 2 3 4 5]
```

In [35]:
```
# 2D Array
arr2 = np.array([[1, 2, 3], [4, 5, 6]])
print("2D Array:", arr2)
```

```
2D Array: [[1 2 3]
 [4 5 6]]
```

In [36]:
```
#  STEP 3: Array Properties
print("Shape:", arr2.shape)
print("Dimensions:", arr2.ndim)
print("Data Type:", arr2.dtype)
```

```
Shape: (2, 3)
Dimensions: 2
Data Type: int32
```

In [38]:
```
# STEP 4: Special Arrays
zeros = np.zeros((2,3))
ones = np.ones((2,3))
identity = np.eye(3)

print("Zeros:", zeros)
print("Ones:", ones)
print("Identity Matrix:", identity)
```

```
Zeros: [[0. 0. 0.]
 [0. 0. 0.]]
Ones: [[1. 1. 1.]
 [1. 1. 1.]]
Identity Matrix: [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

In [39]:
```
# STEP 5: Array Operations
arr = np.array([10, 20, 30, 40])
print("Addition:", arr + 5)
print("Multiplication:", arr * 2)
print("Square:", arr ** 2)
```

```
Addition: [15 25 35 45]
Multiplication: [20 40 60 80]
Square: [ 100  400  900 1600]
```

In [40]:
```
#  STEP 6: Mathematical Functions
print("Mean:", np.mean(arr))
print("Median:", np.median(arr))
print("Standard Deviation:", np.std(arr))
```

```
Mean: 25.0
Median: 25.0
Standard Deviation: 11.180339887498949
```

In [41]:
```
#  STEP 7: Indexing & Slicing
print("First element:", arr[0])
print("Slice:", arr[1:3])
```

```
First element: 10
Slice: [20 30]
```

In [43]:
```
#  STEP 8: Reshaping
arr3 = np.arange(1, 10)
reshaped = arr3.reshape(3,3)
print("Array",arr3)
print("Reshaped:", reshaped)
```

```
Array [1 2 3 4 5 6 7 8 9]
Reshaped: [[1 2 3]
 [4 5 6]
 [7 8 9]]
```

In [44]:
```
#  STEP 9: Random Numbers
rand = np.random.rand(3,3)
print("Random:", rand)
```

```
Random: [[0.42714771 0.26519545 0.92042356]
 [0.70625982 0.19858691 0.23996524]
 [0.61340165 0.07679773 0.77338076]]
```

In [46]:
```
#  STEP 10: Aggregation
print("Array",arr)
print("Sum:", np.sum(arr))
print("Max:", np.max(arr))
print("Min:", np.min(arr))
```

```
Array [10 20 30 40]
Sum: 100
Max: 40
Min: 10
```

In [47]:
```
print("NumPy Lab Completed!")
```

```
NumPy Lab Completed!
```

==========

PANDAS LAB (Python for Data Science - Part 3)

==========

In [54]:
```
#  STEP 1: Import Pandas
import pandas as pd
```

In [50]:
```
# STEP 2: Create DataFrame
# From dictionary
student_data = {
    'Name': ['A', 'B', 'C', 'D'],
    'Age': [20, 21, 19, 22],
    'Marks': [85, 90, 78, 92]
}
```

In [51]:
```
student_data
```

Out[51]:
```
{'Name': ['A', 'B', 'C', 'D'],
 'Age': [20, 21, 19, 22],
 'Marks': [85, 90, 78, 92]}
```

In [62]:
```
df = pd.DataFrame(student_data)
```

In [64]:
```
df
```

Out[64]:

	Name	Age	Marks
0	A	20	85
1	B	21	90
2	C	19	78
3	D	22	92

In [66]:
```
# STEP 3: Basic Inspection
df.head()
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    4 non-null      object
 1   Age     4 non-null      int64
 2   Marks   4 non-null      int64
dtypes: int64(2), object(1)
memory usage: 228.0+ bytes
```

Out[66]:

	Age	Marks
count	4.000000	4.000000
mean	20.500000	86.250000
std	1.290994	6.238322
min	19.000000	78.000000
25%	19.750000	83.250000
50%	20.500000	87.500000
75%	21.250000	90.500000
max	22.000000	92.000000

In [67]:
```
df['Marks']
```

Out[67]:
```
0    85
1    90
2    78
3    92
Name: Marks, dtype: int64
```

In [69]:
```
df[['Name','Marks']]
```

Out[69]:

	Name	Marks
0	A	85
1	B	90
2	C	78
3	D	92

In [71]:
```
#df.iloc[row_index, column_index]
df.iloc[0] # selects First row
```

Out[71]:
```
Name      A
Age      20
Marks    85
Name: 0, dtype: object
```

In [73]:
```
df.iloc[0:3]   #Select Multiple Rows
```

Out[73]:

	Name	Age	Marks
0	A	20	85
1	B	21	90
2	C	19	78

In [75]:
```
df.iloc[0,1] # intestsection element of first and second rows
```

Out[75]:
```
20
```

In [77]:
```
df.iloc[:,1] # selects entire  second column
```

Out[77]:
```
0    20
1    21
2    19
3    22
Name: Age, dtype: int64
```

In [79]:
```
df.iloc[:,0:2] #Select Eintire Multiple Columns
```

Out[79]:

	Name	Age
0	A	20
1	B	21
2	C	19
3	D	22

In [81]:
```
df
```

Out[81]:

	Name	Age	Marks
0	A	20	85
1	B	21	90
2	C	19	78
3	D	22	92

In [86]:
```
df.iloc[0:2,0:2]       # intersection elements of Rows 1st ,2nd  and columns 2nd ,3
```

Out[86]:

	Name	Age
0	A	20
1	B	21

In [87]:
```
df.iloc[-1]          # Select Last Row
```

Out[87]:
```
Name      D
Age      22
Marks    92
Name: 3, dtype: object
```

In [90]:
```
#  STEP 5: Add / Modify Columns
df['Grade'] = ['B','A','C','A']
df
```

Out[90]:

	Name	Age	Marks	Grade
0	A	20	85	B
1	B	21	90	A
2	C	19	78	C
3	D	22	92	A

In [91]:
```
# Modify values
df['Marks'] = df['Marks'] + 5
```

In [92]:
```
df
```

Out[92]:

	Name	Age	Marks	Grade
0	A	20	90	B
1	B	21	95	A
2	C	19	83	C
3	D	22	97	A

In [95]:
```
# 🔷 STEP 6: Filtering Data
high_marks = df[df['Marks'] > 85]
high_marks
```

Out[95]:

	Name	Age	Marks	Grade
0	A	20	90	B
1	B	21	95	A
3	D	22	97	A

In [98]:
```
# Create sample with missing
sample = pd.DataFrame({
    'A':[1,2,None,4],
    'B':[5,None,7,8]
})

sample.isnull().sum()
```

Out[98]:
```
A    1
B    1
dtype: int64
```

In [101…
```
# Fill missing
sample['A'] = sample['A'].fillna(sample['A'].mean())
sample['B'] = sample['B'].fillna(0)

sample
```

Out[101]:

	A	B
0	1.000000	5.0
1	2.000000	0.0
2	2.333333	7.0
3	4.000000	8.0

In [102…
```
# 🔷 STEP 9: GroupBy Operation
group = df.groupby('Grade')['Marks'].mean()
print("Average Marks by Grade:", group)
```

```
Average Marks by Grade: Grade
A    96.0
B    90.0
C    83.0
Name: Marks, dtype: float64
```

In [104…]
```
# STEP 10: Sorting
sorted_df = df.sort_values(by='Marks', ascending=False)
sorted_df
```

Out[104]:

	Name	Age	Marks	Grade
3	D	22	97	A
1	B	21	95	A
0	A	20	90	B
2	C	19	83	C

In [105…]
```
print("Pandas Lab Completed!")
```

```
Pandas Lab Completed!
```

==========

DATA VISUALIZATION LAB (Part 4)

==========

In [106…]
```
import matplotlib.pyplot as plt
import seaborn as sns
```

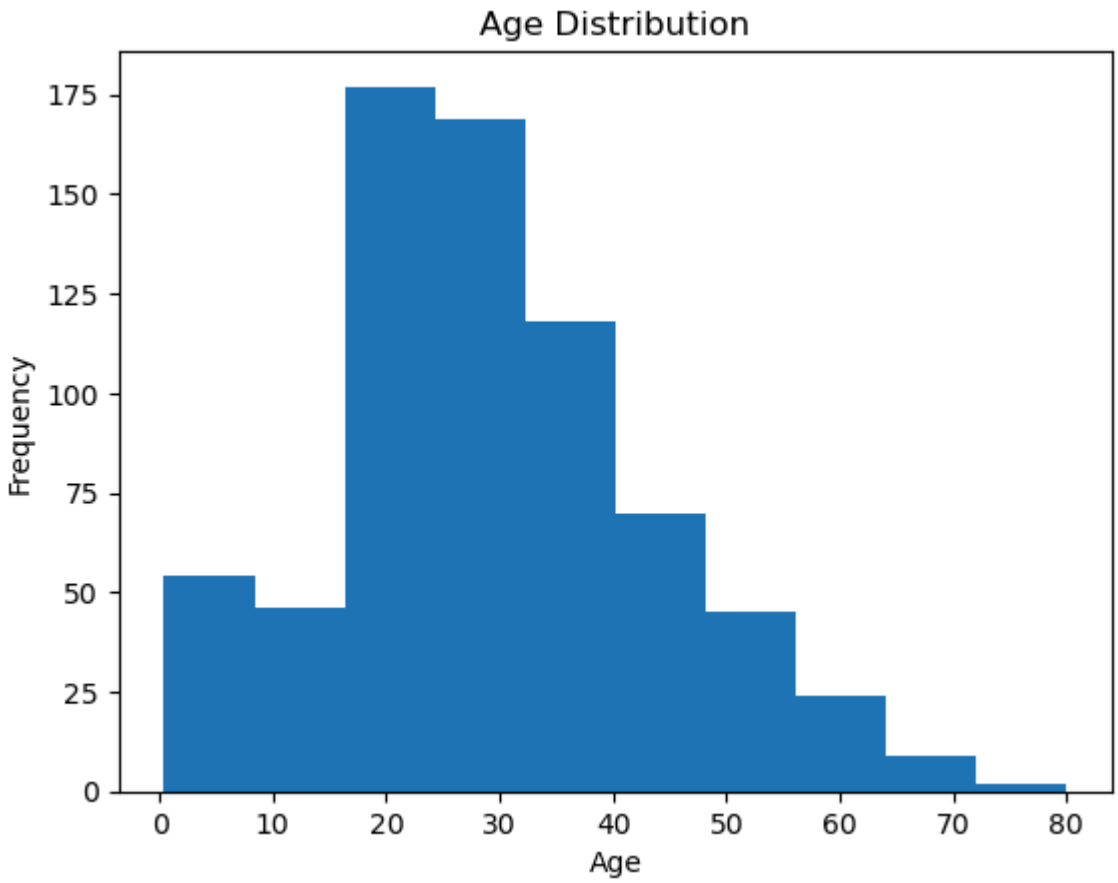
In [107…]
```
# Use seaborn dataset for visualization
viz_df = sns.load_dataset('titanic')
```

In [111…]
```
viz_df.head()
```

Out[111]:

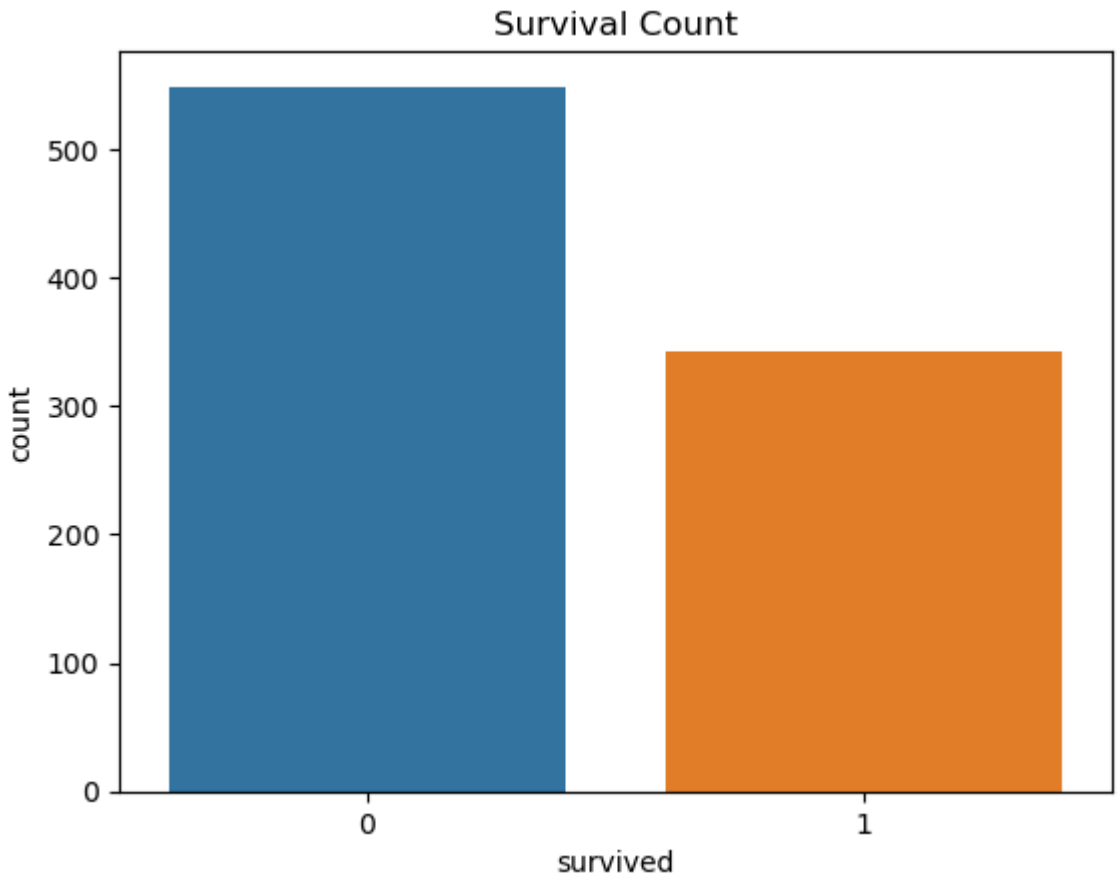
	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	de
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	Na
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	Na
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	Na

In [108…]
```
# 🔷 STEP 2: Histogram
plt.figure()
plt.hist(viz_df['age'].dropna())
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```



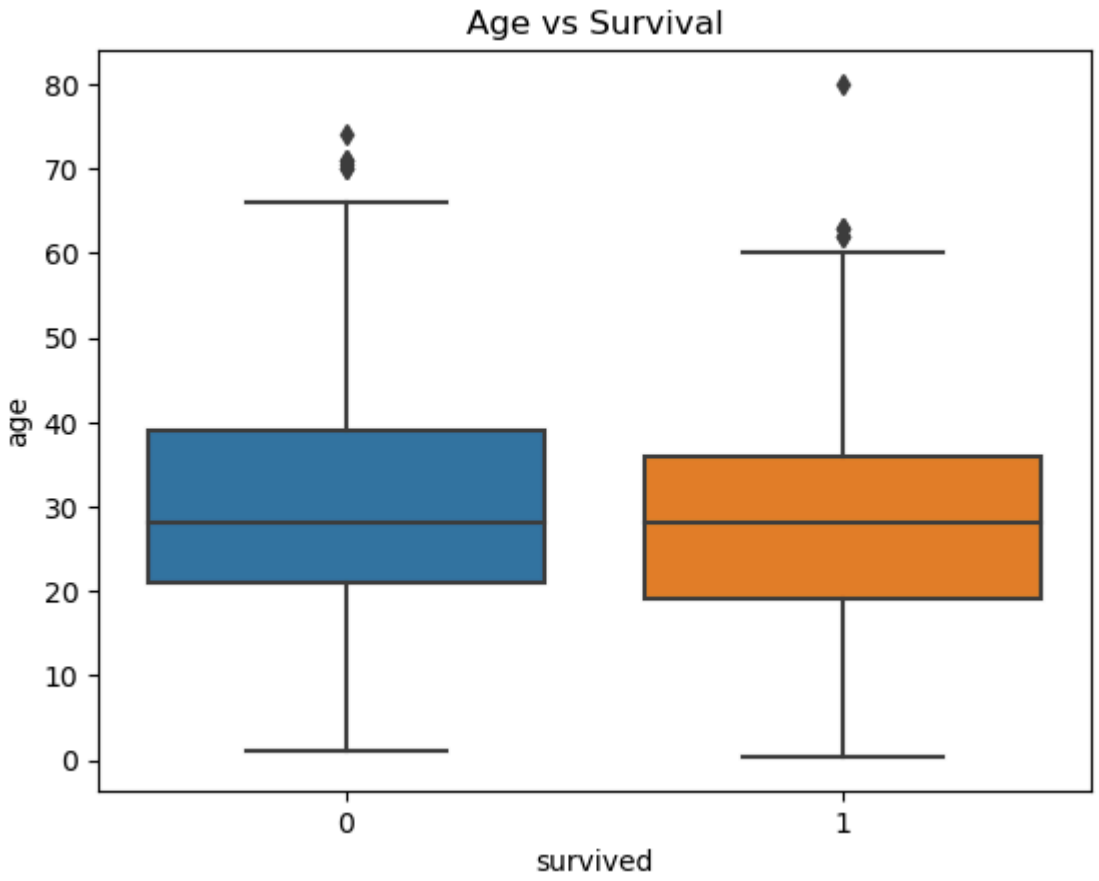
In [109…

```
#  STEP 3: Count Plot
plt.figure()
sns.countplot(x='survived', data=viz_df)
plt.title("Survival Count")
plt.show()
```



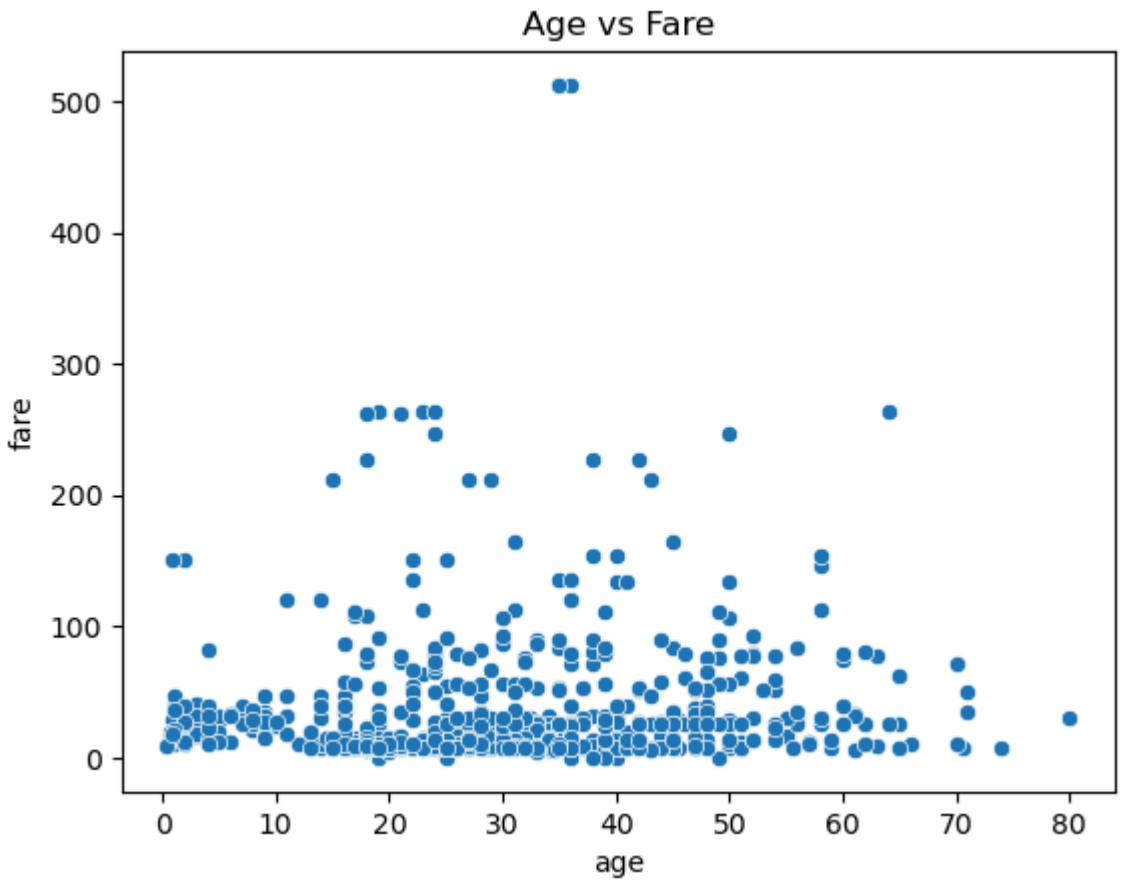
In [112…

```
# STEP 4: Box Plot
plt.figure()
sns.boxplot(x='survived', y='age', data=viz_df)
plt.title("Age vs Survival")
plt.show()
```



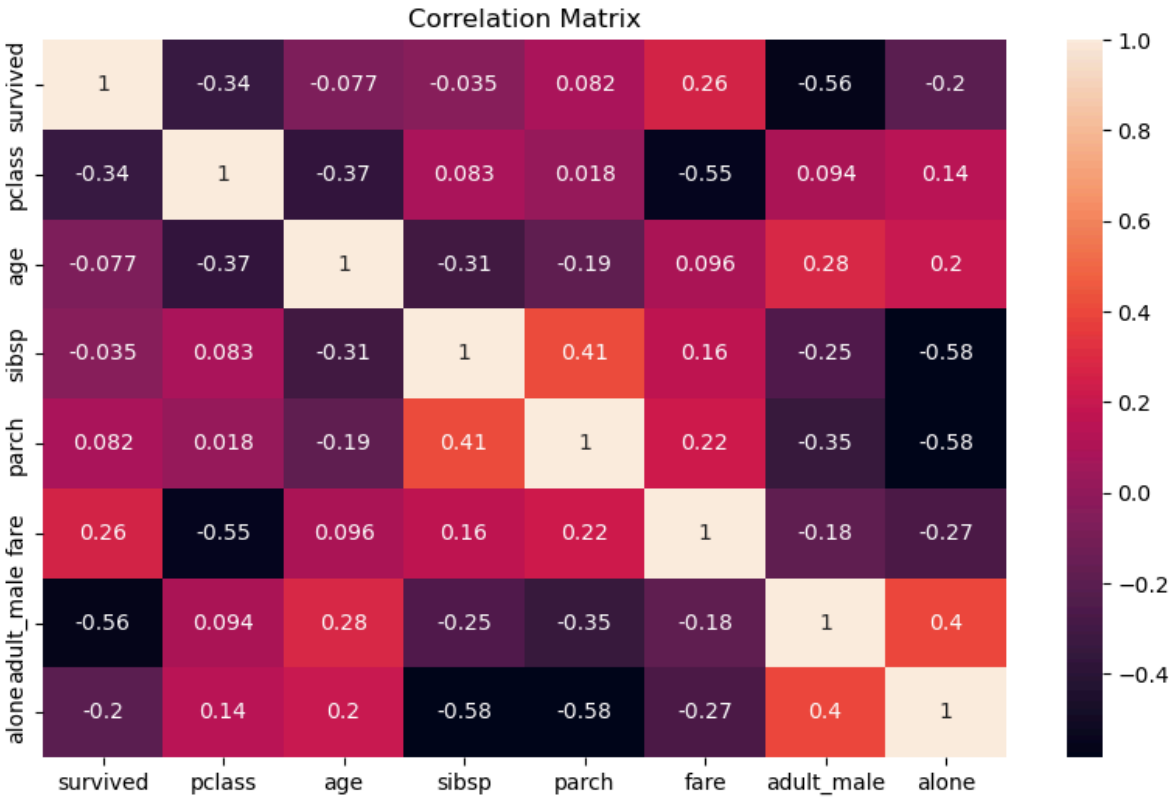
In [113…

```
#  STEP 5: Scatter Plot
plt.figure()
sns.scatterplot(x='age', y='fare', data=viz_df)
plt.title("Age vs Fare")
plt.show()
```



In [114…

```
#  STEP 6: Correlation Heatmap
plt.figure(figsize=(10,6))
sns.heatmap(viz_df.corr(numeric_only=True), annot=True)
plt.title("Correlation Matrix")
plt.show()
```



In [ ]: